Subfix: the six models we trained

Audio-aware subtitle correction. All figures are measured on held-out data, from the repository's metrics files.

#	Model	Type	Used for	Parameters	Dataset	Training	Result
1	**Language model**	Kneser-Ney 4-gram (counts, no neural net)	Scores how fluent a word is in context, to choose between subtitle and Whisper words	13.4M n-grams (55,757 words)	LibriSpeech-PC book text: 563k sentences, 8.5M words	Counting + smoothing, CPU, ~2 min	Perplexity 284 (dev-clean)
2	**Punctuation tagger**	Word + casing + character-CNN, 2-layer BiLSTM	Adds . , ? to words that have none	12.3M	LibriSpeech-PC text, 8.5M words (50% lower-cased)	Cross-entropy, AdamW, 8 epochs, ~41 min, GPU	F1 0.81 cased / 0.67 lowercase
3	**Error corrector**	Transformer encoder-decoder (4+3 layers), own BPE tokenizer	Rewrites broken sentences (typos, homophones, dropped or doubled words)	8.4M	513k LibriSpeech sentences, made noisy on purpose (synthetic + real Whisper-tiny errors)	Seq2seq, label smoothing 0.1, 15 epochs, ~76 min, GPU	Error rate 13.9% to 4.3%; 97.6% of clean text unchanged
4	**POS tagger**	Averaged perceptron	Finds good line-break points (never split an article from its noun)	127k features	Brown corpus: 54,473 sentences (87 tags mapped to 17)	5 passes, CPU, ~3 min	Accuracy 97.2% (held out)
5	**CTC aligner**	Log-mel, 2 conv layers, 4-layer BiLSTM, CTC loss	Gives each word an exact start time (forced alignment)	9.3M	LibriSpeech train-clean-100: 100 h, 251 speakers	CTC, AdamW, SpecAugment, fp16, best epoch 11, ~3 h, GPU	Word-start error 25 ms; character error 7.8%
6	**Speaker model**	Dilated-conv network with attentive pooling, angular-margin softmax	Tells speakers apart to label subtitles SPEAKER_01 / SPEAKER_02 (opt-in)	1.2M	LibriSpeech train-clean-100: 251 speakers, random 2 s crops, noise added	251-way classification, AdamW, 12 epochs planned, GPU	Verification EER 5.05% on unseen speakers

Same recipe for the neural models (2, 3, 5, 6)

A small network, a loss, the AdamW optimiser, a warm-up then decaying learning rate, gradient clipping, and a separate dev set that picks the best epoch. Every trainer saves a checkpoint each epoch and can resume. All training ran on one laptop (RTX 4050, 6 GB GPU, 16 GB RAM).

Why train our own instead of using pretrained models

To control and understand every part, to run offline on a small GPU, and to keep the system small: these six models total about 31M neural parameters, against roughly 1.2 billion for the pretrained models they replaced. Accuracy did not drop: pipeline error rate went from 3.03% to 2.75% and cue-start timing error from 50 to 39 ms.

The one pretrained model

Whisper (large-v3-turbo, trained by OpenAI, about 0.8 billion parameters) does the speech recognition. We did not train it. Everything else in the pipeline is ours.

Where the code is

Trainers: `experiments/train_lm.py, train_punct.py, train_corrector.py, train_pos.py, train_aligner.py, train_speaker.py`. Model definitions: `src/subfix/` (ngram.py, textfix/punct.py, textfix/seq2seq.py, bpe.py, postag.py, acoustic.py, speaker.py). Trained files: `models/`.